instituto superior de engenharia de lisboa

Trabalho de Projeto

Relatório do Projeto "`Dado Eletrónico`"

Trabalho realizado por:

Gustavo Costa | Nº 52808

Rafael Pereira | Nº 52880

Turma: **LEIC24D**

Docente: **João Patriarca**

Licenciatura em Engenharia Informática e de Computadores

Arquitetura de Computadores (AC)

2024 / 2025 - Semestre de Verão

Conteúdo

1 Introdução **1**

2 Análise do Problema **2**
2.1 Entradas e Saídas Relevantes 2
2.2 Requisitos Temporais e de Funcionamento 2
2.3 Sincronização e Resposta a Eventos Assíncronos 3
2.4 Decisões de Projeto e Resolução de Ambiguidades 3

3 Solução Desenvolvida **5**
3.1 Implementação de Hardware 5
3.2 Mapa de Endereçamento 5
3.3 Organização do Software 6

4 Testes e Resultados **8**

5 Respostas às Questões do Enunciado **9**
5.1 1.ª Questão 9
5.2 2.ª Questão 9
5.3 3.ª Questão 10
5.4 4.ª Questão 10

6 Conclusões **11**

7 Referências **12**

A Codigo Fonte do Trabalho **13**

Autores: Gustavo Costa: 52808 / Rafael Pereira: 52880

1 Introdução

Este relatório descreve o desenvolvimento de um sistema embebido baseado no processador `P16`, concebido para simular o funcionamento de um dado eletrónico configurável com 4, 6, 8 ou 12 faces, conforme definido no enunciado do Trabalho de Projeto da UC de Arquitetura de Computadores (AC).

O projeto foi implementado utilizando o conjunto das placas `SDP16` e `ATB`, um circuito `FED` (*Falling Edge Detector*) e um *Display* de 7 segmentos `(7SEG-Break)`, como se pode verificar na Figura 1, e tem como principal objetivo a aplicação integrada de conhecimentos práticos nas áreas de mapeamento de memória, gestão de interrupções externas, mecanismos de temporização com precisão e programação em linguagem *assembly* do `P16`.

Os objetivos funcionais do sistema incluem a geração de números pseudoaleatórios em função do número de faces selecionado pelo jogador, a simulação visual do lançamento do dado através de efeitos dinâmicos num *display* de 7 segmentos e a resposta eficaz a eventos assíncronos, como a ativação de um interruptor físico. O sistema deve ainda cumprir requisitos temporais rigorosos, nomeadamente um efeito luminoso com duração mínima de 1 segundo e um intervalo de espera de 10 segundos após a apresentação do resultado, antes de permitir um novo lançamento.

A estrutura deste relatório segue as orientações definidas no guia para a elaboração do relatório da unidade curricular, disponibilizado na plataforma *Moodle*. Inicia-se com uma análise detalhada do problema, na qual são identificados os requisitos funcionais do sistema. Segue-se a descrição da solução desenvolvida, abordando a organização modular do *software*. A secção de testes e resultados documenta a estratégia de validação experimental e os principais dados obtidos. As respostas às questões do enunciado fundamentam tecnicamente as decisões de conceção, enquanto a secção final de conclusões sintetiza o grau de cumprimento dos objetivos definidos inicialmente.

Cada secção corresponde, assim, a uma etapa do processo de desenvolvimento, resultando num documento claro e tecnicamente rigoroso.

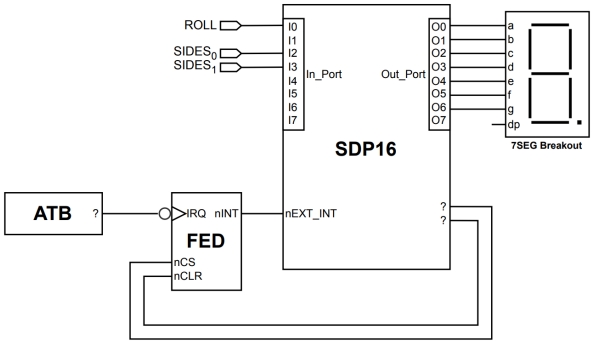

Figura 1: Diagrama de blocos do protótipo do sistema

2 Análise do Problema

O sistema a desenvolver tem como objetivo principal a simulação de um dado eletrónico configurável, com suporte para quatro tipos distintos de dados: 4 faces, 6 faces, 8 faces e 12 faces.

A seleção do tipo de dado é feita dinamicamente através de interruptores 2 e 3, físicos na placa `SDP16`. O comportamento do sistema está condicionado por requisitos funcionais e temporais bem definidos, cuja análise detalhada é essencial para garantir uma implementação coerente e funcional.

Como, de acordo com o enunciado, o *display* de 7 segmentos apenas pode mostrar números de 1 a 9, o que suscita dúvida na hora de definir as faces do dados, quando este é o dado de 12 faces. Para os outros 3 tipos de dados (4, 6 e 8 faces) foram usados nas faces, os números de 1 a N, onde N é o número de faces do dado, e com uma distribuição equilibrada (1 número por face do dado). Já para o dado de 12 faces, como o número de faces é superior ao número de dígitos que o *display* de 7 segmentos pode mostrar, foi necessário arranjar uma alternativa.

A alternativa escolhida poderia ter diferentes abordagens, porém optou-se pelos dígitos de 1 a 6, aplicando-os a 2 faces cada, mantendo o dado equilibrado e não viciado, a nível de probabilidades. Com isto, para todos os tipos de dados, temos dados equilibrados tal como era pedido no enunciado e, também, já resolvida a questão do dado de 12 faces.

2.1 Entradas e Saídas Relevantes

O sistema interage com o jogador através de periféricos ligados aos portos de entrada e saída do processador `P16`.

As entradas são as seguintes:

- `ROLL` (DIP-Switch SW1[0]): Interruptor utilizado para acionar o lançamento do dado. Detetado através de uma transição descendente, monitorizada com recurso ao circuito FED.
- `SIDES[1:0]` (DIP-Switch SW1[2] e [3]): Seleção do tipo de dado a utilizar. A combinação dos dois bits permite codificar 4, 6, 8 ou 12 faces, associadas às combinações '`00`', '`01`', '`10`' e '`11`', respetivamente.

E a saída relevante é:

- *Display* de 7 segmentos: Representação do número sorteado após o lançamento. A codificação dos dígitos é feita com base numa tabela `hex27seg` que mapeia os valores numéricos para os padrões de segmentos desejados.

2.2 Requisitos Temporais e de Funcionamento

O sistema deve garantir o cumprimento rigoroso dos seguintes requisitos de temporização:

- Efeito luminoso (animação): Deve durar pelo menos 1 segundo, independentemente do tipo de dado escolhido. A

animação é composta por uma sequência de padrões que simulam o movimento do dado no *display*. Optou-se por uma animação em forma de '8'.

- Intervalo de espera após lançamento: Após a apresentação do número sorteado, o sistema entra num estado de espera de 10 segundos, durante os quais ignora novos eventos no interruptor correspondente ao sinal `ROLL`, prevenindo ativações repetidas acidentais.

Estes requisitos são garantidos com o uso do circuito `FED`, cuja frequência é controlada através do circuito `OSCILLATOR` da placa ATB, que permite usar as frequências de `1Hz`, `10Hz`, `1kHz` e `100KHz`. Para as funcionalidades do nosso programa, concluímos que a melhor frequência a ser usada é a de `10Hz`, pela razão que irá ser apresentada posteriormente. A contagem de eventos (*ticks*) do `FED` é utilizada para implementar temporizadores no *software*, com base na variável `tick_cnt` atualizada na rotina de interrupção externa (`extint_vector`). A rotina `wait_ticks` permite realizar esperas parametrizadas, abstraindo a contagem manual de ciclos.

Os parâmetros de temporização definidos são:

- `FRAME_TICKS` = 1 (correspondente a `100 ms`)
- `FRAMES_ROLL` = 30 (30 iterações de `100 ms` = 3 segundos de animação)
- `END_ROLL` = 100 (espera final de 10 segundos)

Embora o enunciado exija uma animação de pelo menos 1 segundo, a implementação excede este mínimo (3 segundos), mas garante-se uma maior expressividade visual, sendo possível observar vários ciclos dos passos da animação, perfazendo uma sequência suave e constante.

2.3 Sincronização e Resposta a Eventos Assíncronos

O programa foi feito para reagir de forma correta e coerente, a eventos externos, nomeadamente a transição descendente do sinal `ROLL`, detetada pelo circuito *Falling Edge Detector* (`FED`). Este periférico emite um sinal de interrupção externa sempre que ocorre uma transição de nível alto para baixo no sinal `ROLL`.

A rotina de interrupção (`extint_vector`) é responsável por confirmar e limpar o evento no `FED` e incrementar a variável global `tick_cnt`, que serve como base para todas as operações de temporização no sistema.

Desta forma, o sistema consegue detetar eventos assíncronos com precisão sem recorrer a *polling* contínuo, permitindo um controlo mais eficaz dos tempos.

2.4 Decisões de Projeto e Resolução de Ambiguidades

Durante a leitura e interpretação do enunciado, surgiram diversas situações que exigiram decisões de projeto para garantir o correto funcionamento e a coerência da so-

lução:

- Representação dos diferentes tipos de dados: A seleção `SIDES[1:0]` codifica o tipo de dado em 2 bits. Para traduzir esta informação num número de faces (4, 6, 8, 12), foi criada uma tabela (`max_sides`) indexada pelo valor de `SIDES` que mapeia diretamente para o número de faces correspondente. Esta abordagem permite simplificar o código e evitar instruções de comparação redundantes.

- Geração de números pseudoaleatórios: Foi implementado um gerador de números pseudoaleatórios em *assembly*, disponibilizado pelos docentes da disciplina [7], garantindo compatibilidade total com a `ISA` do `P16`. A operação de módulo foi realizada com subtrações sucessivas para `r0 mod N`, dado que $N \leq 12$ e, portanto, o ciclo é garantidamente rápido e eficiente.

- Gestão da mudança de tipo de dado: Para garantir que a alteração do tipo de dado em `SIDES` resulta imediatamente numa mudança visível no *display*, foi implementada a função `action`, que verifica se o índice do dado mudou e, nesse caso, atualiza a saída com um valor novo.

- Estratégia de animação: A animação foi concebida como uma sequência cíclica de padrões no *display* de 7 segmentos. A sequência está definida na constante `anim_sequence`, com 8 valores (máximo da animação) que são apresentados a cada `100 ms`.

- Comportamento em repouso: O sistema entra num ciclo de espera e apenas reage quando o interruptor relativo ao `ROLL` vai para `ON` e para `OFF`, e com a transição descendente, assegura que o lançamento do dado não ocorre por engano.

Esta análise serviu de base sólida para a definição da solução técnica descrita na secção seguinte, garantindo que todas as especificações do enunciado são interpretadas de forma rigorosa e aplicadas de modo consistente na implementação.

3 Solução Desenvolvida

Esta secção descreve, de forma estruturada, a solução proposta para o sistema de dado eletrónico baseado no processador `P16`. A implementação foi orientada pelos requisitos funcionais e temporais identificados na análise do problema, assegurando o mapeamento de memória e a organização modular do programa.

3.1 Implementação de Hardware

A implementação de hardware foi realizada com base nas placas fornecidas no laboratório: `SDP16`, `FED`, *Display* de 7 segmentos e `ATB`. A ligação entre os diferentes componentes segue o esquema proposto no enunciado (Figura 1), respeitando as restrições do sistema e as interfaces disponíveis.

- A placa `SDP16` integra o processador `P16`, um porto de entrada (`INPORT`) e um porto de saída (`OUTPORT`) mapeados em memória.
- O sinal `ROLL` (DIP-Switch [0]) e os sinais `SIDES[1:0]` (DIP-Switch [2] e [3]) são lidos pelo processador através do porto de entrada.
- A comunicação com o *display* de 7 segmentos é feita através do porto de saída, usando os bits `O0` a `O6` para controlar os segmentos de '`a`' a '`g`'.

O circuito `FED` é responsável por detetar transições descendentes no sinal `ROLL`, sinalizando uma interrupção externa no pino `nEXT_INT` do `P16`. O `FED` é também usado como fonte de temporização, gerando impulsos periódicos a partir de um *clock* fornecido pela placa `ATB`. Estes impulsos são usados para atualizar a variável `tick_cnt`, base para os temporizadores do sistema.

Para permitir esta integração, o `FED` foi mapeado no espaço de endereços do processador (ver Secção 3.2) e configurado de modo a responder corretamente à ativação do interruptor. O sinal de interrupção externa é ativado quando o jogador move o interruptor `ROLL` da posição `ON` para `OFF`, originando uma transição descendente no sinal. Esta transição é captada pelo `FED`, que gera o impulso necessário na linha `IRQ`.

A descodificação de endereços adotada assegura que cada dispositivo acede ao seu espaço sem sobreposição. A utilização de periféricos adicionais (como o `FED`) obrigou à definição de regiões específicas fora do espaço ocupado pela memória principal e pelos periféricos base.

3.2 Mapa de Endereçamento

O espaço de endereços do processador `P16` foi estruturado para incluir, além da memória e dos periféricos base, o circuito `FED`. O mapa de endereçamento representado na Figura 2, tem como propósito satisfazer todas as necessidades ao nível da memória da `SDP16`, gerindo bem os endereços a usar e com estratégias que foram apuradas, relativamente, a mapas de endereçamento, apuradas em aula. Apresenta-se, em seguida, o referido mapa de endereçamento:

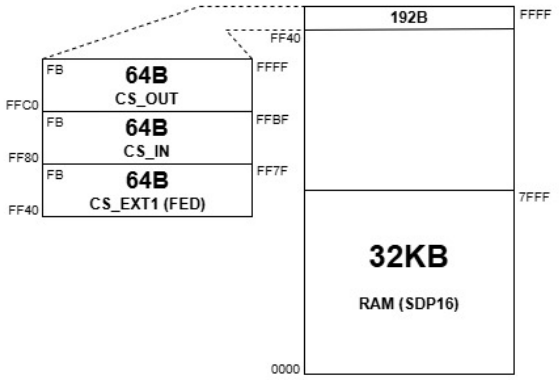

Figura 2: Mapa de endereçamento relativo à nossa implementação

A definição destes endereços foi realizada tendo em consideração:

- A minimização de conflitos entre periféricos;
- A compatibilidade com o código *assembly*, que utiliza acesso direto (via instruções `ldr`, `strb`, etc.);

A região definida pelo endereço `0xFF40` foi atribuída ao `FED` por convenção e para evitar sobreposição com os endereços já ocupados pelos portos padrão do `P16` [8]. Essa escolha é refletida nas instruções de limpeza de interrupção dentro da rotina `extint_vector`.

O mapeamento de periféricos adicionais foi feito com acesso direto via registos de 16 bits e literais constantes, assegurando desempenho e legibilidade no código *assembly*.

3.3 Organização do Software

A organização do software desenvolvido para este sistema foi concebida de forma modular, promovendo a separação de responsabilidades e a legibilidade do código, escrito integralmente em linguagem assembly compatível com a `ISA` do processador `P16`. A lógica principal do programa assenta numa estrutura em camadas, onde se distinguem rotinas de acesso direto aos periféricos, rotinas de processamento funcional e uma camada de controlo que coordena o comportamento do sistema.

Na base do programa encontram-se as rotinas de interface com o hardware. As funções `inport_read` e `outport_write` permitem o acesso direto ao porto de entrada e ao porto de saída, respetivamente, através de instruções `ldr` e `strb`, utilizando endereços absolutos definidos com diretivas `.equ`. Estes endereços (`INPORT_ADDRESS`, `OUTPORT_ADDRESS` e `FED_ADDRESS`) refletem parte do mapeamento de memória apresentado na secção anterior. A leitura das entradas do DIP-switch é essencial para identificar o sinal `ROLL` (bit 0) e a configuração dos bits `SIDES0` e `SIDES1` (bits 2 e 3), utilizados para determinar o tipo de dado a lançar. A escrita no *display* de 7 segmentos é feita com base nos valores contidos na tabela `hex27seg`, mapeando diretamente os dígitos para os padrões binários adequados.

A temporização é assegurada com base no circuito `FED`, que gera interrupções periódicas captadas pela rotina `extint_vector`. Esta rotina de interrupção incrementa a variável global `tick_cnt`, servindo de base para a função `wait_ticks`, que implementa atrasos parametrizados em *ticks*. Esta abordagem garante o cumprimento dos requisitos temporais do enunciado, como os 10 segundos de espera entre lançamen-

tos (`END_ROLL` = 100) e a duração mínima de 1 segundo para o efeito luminoso (`FRAME_TICKS` × `FRAMES_ROLL` = 1 segundo ou mais).

A camada funcional intermédia do programa inclui o sistema de geração de números pseudoaleatórios, essencial para simular o lançamento do dado. A função `rand` implementa um gerador congruencial linear (LCG), utilizando aritmética de 32 bits e multiplicação com a rotina auxiliar `umull32`. A semente (`seed`) pode ser alterada com a função `srand`, embora neste projeto seja inicializada de forma estática. O número aleatório gerado é convertido para um valor dentro do intervalo válido para o tipo de dado selecionado através da rotina `roll`, que usa a função `getDiceMax` e uma operação de redução por subtração iterada (módulo).

A camada de controlo do programa é coordenada pela rotina `main`, responsável pela inicialização do sistema e pelo ciclo principal de espera e resposta a eventos. Inicialmente, o programa ativa as interrupções externas (`ENABLE_EXTINT`) e realiza a leitura do valor atual de `SIDES` para exibir uma face aleatória do dado logo à partida. O ciclo principal (`main_start_loop` e `Main_loop`) espera por uma alteração no estado do sinal `ROLL`, detetando transições através da comparação entre leituras sucessivas. Quando é identificado um novo lançamento, o programa atualiza o tipo de dado, executa a animação (`anim`), gera o número aleatório e escreve o resultado no *display*. Após isso, entra num período de espera (`wait_ticks`) correspondente aos 10 segundos exigidos antes de aceitar um possível novo lançamento.

A função `action` permite detetar alterações no tipo de dado durante o estado de espera. Se os bits `SIDES[1:0]` forem alterados, esta função atualiza o novo intervalo de faces possíveis relativas ao novo dado selecionado, porém não mostra o valor no *display*, mantendo-se representada neste, a face anteriormente sorteada até ser executado um novo `ROLL`. Esta lógica garante a interatividade contínua do sistema sem a necessidade de reiniciar o programa.

Por fim, o efeito visual de lançamento do dado é gerado pela função `anim`, que recorre a uma sequência cíclica de padrões (`anim_sequence`) mostrados no *display* da placa `7SEG Breakout` com um intervalo temporal de `100 ms` entre cada atualização. A animação repete-se por um número de iterações definido em `FRAMES_ROLL`, proporcionando um efeito visual fluido e perceptível, parecido ao comportamento de um dado real a rodar.

4 Testes e Resultados

A validação do sistema foi realizada em duas fases complementares: simulação com recurso ao depurador `p16dbg` e teste experimental em laboratório, utilizando as placas e circuito já citados, conforme a arquitetura definida no enunciado.

Durante a fase de simulação, foi possível verificar o correto funcionamento da lógica principal do programa, incluindo a leitura dos sinais `ROLL` e `SIDES`, a execução da rotina `roll` e a escrita no `display` com base na sequência definida na tabela `hex27seg`.

A simulação permitiu ainda testar o gerador pseudoaleatório (`rand`) e a função de temporização `wait_ticks`, validando a sua evolução com base no incremento da variável `tick_cnt`.

A fase de teste em ambiente real permitiu validar a resposta assíncrona do sistema à transição descendente no sinal `ROLL`, gerada pelo DIP-switch SW1 e captada através do circuito `FED`. Confirmou-se que a rotina de interrupção externa (`extint_vector`) é invocada corretamente e que o tempo entre chamadas é consistente com *clock* aplicado ao `FED` (`10Hz`). A variável `tick_cnt` foi incrementada conforme esperado, confirmando o bom funcionamento da temporização.

Durante os testes foram verificados os seguintes aspetos essenciais:

- A duração da animação no *display* é superior a 1 segundo, como exigido.
- O sistema ignora eventos de `ROLL` durante os 10 segundos de espera, validando o comportamento especificado no enunciado.
- A alteração de `SIDES` durante o estado de espera é detetada e reflete-se corretamente no próximo dígito sorteado e apresentado.

Durante os testes não foram observados comportamentos incorretos nem efeitos indesejados. O sistema demonstrou ser funcional e alinhado com os objetivos definidos.

5 Respostas às Questões do Enunciado

5.1 1.ª Questão

A questão colocada foi: "`Explique a solução adotada para ligar o circuito FED à placa SDP16`".

A questão colocada foi: "`Explique a solução adotada para ligar o circuito FED à placa SDP16`".

O circuito `FED` deteta transições descendentes ($1 \rightarrow 0$), funcionando como adaptador entre sinais ativos a nível e sinais ativos a flanco. Neste projeto, é utilizado para detetar o evento de lançamento do dado, através do sinal `ROLL` (I0 do DIP-switch).

A saída `nINT` do `FED` liga-se à entrada de interrupção externa `nEXT_INT` do processador `P16`, invocando a rotina `extint_vector`, responsável por incrementar a variável `tick_cnt`, usada para temporização.

As ligações ao `FED` são feitas da seguinte forma:

A entrada `nCS` liga-se ao sinal `nCS_EXT1`, uma vez que o `FED` está mapeado no endereço `0xFF40` (região `CS_EXT1`).

A entrada `nCLR` é ligada ao sinal `nWRL`, ativado ao escrever no byte inferior de um endereço par.

A entrada `IRQ` recebe o sinal `OSCILLATOR` da placa `ATB`, com frequência de `10Hz`, resultando num *tick* a cada `100ms`.

Estas ligações permitem que o `FED` gere uma interrupção em cada transição descendente e que a sua reposição seja feita por software. A montagem física é feita com *jumpers* entre a breadboard (com o `FED`) e os conectores da placa `SDP16`, conforme ilustrado no Manual de Utilização [2] da mesma.

5.2 2.ª Questão

A questão colocada foi: "`Apresente os cálculos realizados para determinar as temporizações envolvidas no trabalho`".

O sistema usa o circuito `FED` como base de tempo, alimentado por um sinal de `10Hz` proveniente do `OSCILLATOR` da placa `ATB`. Cada *tick* ocorre a cada `100 ms`, e a variável `tick_cnt` é incrementada pela rotina de interrupção externa a cada impulso. Esta variável é usada pela função `wait_ticks` para controlar os atrasos.

As temporizações foram definidas da seguinte forma:

- Animação (efeito visual): A função `anim` executa 30 iterações com 1 *tick* (`100 ms`) entre cada $\rightarrow 30 \times$ `100 ms` $=$ 3 segundos.

- Espera após lançamento: Após mostrar o número, o sistema espera 100 ticks $\rightarrow$ $100 \times$ `100 ms` $= 10$ segundos.

Estes valores foram escolhidos para cumprir, rigorosamente, os requisitos do enunciado, com garantia de haver temporizações precisas com base no sinal fornecido pela `ATB`.

5.3 3.ª Questão

A questão colocada foi: "`Indique, justificando, a latência máxima do sistema no atendimento dos pedidos de interrupção gerados pelo circuito FED`".

A latência máxima é o tempo entre a chegada de um impulso no `FED` (transição descendente no sinal `ROLL`) e o início da execução da rotina `extint_vector`. No `P16`, a deteção de interrupção ocorre no ciclo seguinte ao evento (desde que as interrupções estejam ativas e não se esteja numa instrução crítica), o que equivale a uma latência de, no máximo, 1 ciclo de instrução.

Como o *clock* interno do `P16` é muito superior à frequência do `FED` (`10Hz` $\rightarrow$ 1 interrupção a cada `100 ms`), e cada instrução leva apenas alguns μs , a latência situa-se entre 1 e 10 μs, valor negligenciável comparado com os $1 \cdot 10^5 \mu s$ (`100 ms`) de intervalo entre interrupções.

A escolha do sinal de `10Hz` da `ATB` é adequada: mais rápido que `1Hz` (que tornaria o sistema lento e pouco reativo) e suficientemente espaçado para evitar sobrecarga da `ISR`, ao contrário de `1kHz` ou `100kHz`, que gerariam interrupções a cada 1 ou `0,01 ms`, consumindo recursos e processamento desnecessários.

Assim, `10Hz` oferece um compromisso ideal entre a precisão temporal e eficiência, garantindo que o sistema reaja rapidamente, sem comprometer o desempenho do programa.

5.4 4.ª Questão

A questão colocada foi: "`Indique, justificando, quanto tempo demora, no pior caso, a execução da rotina utilizada para o atendimento da interrupção externa`".

A rotina `extint_vector` é responsável por:

- Guardar os registos `r0` e `r1`;
- Calcular o endereço do `FED` e escrever nesse endereço (limpando a interrupção);
- Ler o valor de `tick_cnt`, incrementá-lo e reescrevê-lo;
- Restaurar os registos `r1` e `r0`;
- Retornar da interrupção com `movs pc, lr`.

A rotina executa cerca de 10 a 12 instruções, todas com complexidade baixa (média de 1 ciclo). Mesmo admitindo que algumas instruções demoram até 2 ciclos, o tempo total de execução situa-se entre 10 e 20 ciclos de instrução.

Com uma estimativa de 1 μs por ciclo, o tempo total de execução da ISR (*Interrupt Service Routine*) é da ordem de 10 a 20 μs.

Este valor representa apenas 0,01% do intervalo entre interrupções (`100 ms`), sendo, portanto, totalmente aceitável. A ISR é curta, eficiente e não bloqueia o sistema, o que permite obter uma resposta determinística e sem riscos de sobreposição de interrupções.

6 Conclusões

O projeto apresentado teve como objetivo o desenvolvimento de um sistema embebido baseado no processador `P16`, capaz de simular o funcionamento de um dado eletrónico configurável com 4, 6, 8 ou 12 faces. Ao longo do desenvolvimento, foram aplicados conhecimentos fundamentais da Unidade Curricular de Arquitetura de Computadores, nomeadamente em gestão de interrupções externas, programação em *assembly*, temporização baseada em *hardware* e organização modular de sistemas.

A solução implementada revelou-se funcional. O sistema deteta corretamente a transição descendente no sinal `ROLL` com o apoio do circuito `FED`, executando uma animação visual coerente no *display* de 7 segmentos e exibindo um número pseudoaleatório conforme o tipo de dado selecionado. A utilização do `FED` como base de tempo e fonte de interrupções permitiu cumprir rigorosamente os requisitos temporais, sem sobrecarga de processamento.

Foram ainda cumpridos todos os aspetos formais solicitados no relatório: mapeamento de *hardware*, cálculo de temporizações, análise da latência da interrupção e justificação do tempo de execução da `ISR`. Os testes realizados, tanto em simulação como em ambiente real, validaram o correto funcionamento da solução.

Conclui-se, assim, que o sistema atinge plenamente os objetivos propostos, constituindo um exercício prático bem-sucedido.

7 Referências

[1] Instituto Superior de Engenharia de Lisboa. *Trabalho de Projeto – Arquitetura de Computadores, 2024/2025*, Versão 2.0.2, Lisboa, 2025.

[2] Instituto Superior de Engenharia de Lisboa, *Manual de Utilização* da Placa SDP16, disponível em: `https://shre.ink/e2s5` (link encurtado), consultado em: 28 abril 2025.

[3] Instituto Superior de Engenharia de Lisboa, *Esquema elétrico (v24.07)* do 7SEG-Breakout, disponível em: `https://shre.ink/e2se` (link encurtado), consultado em: 05 junho 2025.

[4] Instituto Superior de Engenharia de Lisboa, *Manual de consulta rápida das instruções do P16*, disponível em: `https://shre.ink/e2sb` (link encurtado), consultado em: 28 abril 2025.

[5] Instituto Superior de Engenharia de Lisboa, *Guia de codificação das instruções do P16*, disponível em: `https://shre.ink/e2s8` (link encurtado), consultado em: 28 abril 2025.

[6] Instituto Superior de Engenharia de Lisboa, *Sistema de Interrupções do P16 – Atividade Laboratorial #4 (v1.8)*, disponível em: `https://shre.ink/e2sk` (link encurtado), consultado em: 04 junho 2025.

[7] Instituto Superior de Engenharia de Lisboa, *Proposta de implementação do programa pedido no 1.º Trabalho Prático*, disponível em: `https://2425moodle.isel.pt/mod/resource/view.php?id=194201`, consultado em: 04 junho 2025.

[8] Instituto Superior de Engenharia de Lisboa, *19_AC_Interrupções_v2425.pdf*, disponível em: `https://shre.ink/e2BV` (link encurtado), consultado em: 28 abril 2025.

A Codigo Fonte do Trabalho

```
; ====================================================================
;  Ficheiro: TrabalhoProjeto_p16.s
;  Desc:     Projeto de lancamento de dados utiliza "ticks" do FED para
   temporizacao
;            *apenas com instrucoes disponiveis no ISA P16* (sem pseudo-
   operacoes).
;            Substitua o bloco TIMING CONSTANTS se o clock externo
;            no FED for alterado (ex: 100 kHz em vez de 10 kHz).
; --------------------------------------------------------------------
;  BUILD:    p16as -s .stack=0x2000 TrabalhoProjeto_p16.s
; ====================================================================

; ========================== CONSTANTES ==============================
    .equ    INPORT_ADDRESS  , 0xFF80        ; Endereco da porta de
   entrada ( D I P switch )
    .equ    OUTPORT_ADDRESS , 0xFFC0        ; Endereco da porta de saida
    (display de 7 segmentos)
    .equ    FED_ADDRESS     , 0xFF40        ; 1.   byte da regiao
   CS_EXT1 (FED - Hardware Externo para interrupcoes)

    .equ    ENABLE_EXTINT   , 0x10          ; Bit para habilitar
   interrupcoes externas no CPSR

; ------------  Temporizacao derivada do FED  -------------------------
    ; ---------- Constantes de temporizacao ----------
    .equ    FRAME_TICKS     , 1             ; Numero de "ticks" do FED
   por frame da animacao
    .equ    FRAMES_ROLL     , 30            ; Numero de frames durante a
    animacao de lancamento do dado
    .equ    END_ROLL        , 100           ; Duracao em "ticks" para
   exibir o resultado final do lancamento

; ------------  Dados do jogo
   ------------------------------------------
    .equ    MAX_ANIM        , 8             ; Numero de padroes
   distintos na sequencia de animacao
    .equ    TOTAL_SIDES     , 4             ; Numero de tipos de dados
   diferentes (ex: d4, d6, d8, d12)

    .equ    DICE_MSK        , 0x0C          ; Mascara para os bits SIDES
   [1:0] no DIP-switch (selecao do tipo de dado)
    .equ    ROLL_MSK        , 0x01          ; Mascara para o bit ROLL no
    DIP-switch (iniciar lancamento)
```

```
; ------------  RandMax (Limites para gerador de numeros aleatorios)
   ---------------------------------------
    .equ    RAND_MAX_L      , 0xFFFF        ; Parte baixa (16 bits) do
   maior valor (0xFFFFFFFF) para a semente do RNG
    .equ    RAND_MAX_H      , 0xFFFF        ; Parte alta (16 bits) do
   maior valor (0xFFFFFFFF) para a semente do RNG

    .equ    STACK_SIZE      , 64            ; Tamanho da pilha em bytes

; ====================================================================
    .text
    b       reset_vector                    ; 0x0000: Vetor de Reset
    b       extint_vector                   ; 0x0002: Vetor de
   Interrupcao Externa

; ========================== VETORES DE INTERRUPcaO
   ==================================
reset_vector:                               ; Rotina de tratamento do
   vetor de reset
    ldr     sp, _stack_lit                  ; Carrega o ponteiro da
   pilha (sp) com o endereco do topo da pilha
    b       main                            ; Salta para a rotina
   principal do programa
_stack_lit:
    .word   stack_top                       ; Literal que armazena o
   endereco do topo da pilha

extint_vector:                              ; Rotina de tratamento da
   interrupcao externa (endereco 0x0002)
    push    r0                              ; Salva o registador r0 na
   pilha
    push    r1                              ; Salva o registador r1 na
   pilha
    ; Limpa o sinalizador de interrupcao do FED (escrevendo qualquer
   valor no seu endereco)
    mov     r0, #FED_ADDRESS & 0xFF         ; Carrega a parte baixa do
   endereco do FED em r0
    movt    r0, #(FED_ADDRESS >> 8) & 0xFF  ; Carrega a parte alta do
   endereco do FED em r0
    strb    r0, [r0]                        ; Escreve em FED_ADDRESS
   para limpar o pedido de interrupcao
    ; Incrementa o contador global de ticks (tick_cnt)
    ldr     r0, _tick_ptr1                  ; Carrega o endereco da
   variavel tick_cnt em r0
```

```
    ldr     r1, [r0]                        ; Carrega o valor atual de
   tick_cnt em r1
    add     r1, r1, #1                      ; Incrementa o contador de
   ticks
    str     r1, [r0]                        ; Armazena o novo valor de
   tick_cnt
    pop     r1                              ; Restaura o registador r1
   da pilha
    pop     r0                              ; Restaura o registador r0
   da pilha
    movs    pc, lr                          ; Retorna da interrupcao (
   restaura CPSR e PC)

_tick_ptr1: .word tick_cnt                  ; Literal que armazena o
   endereco da variavel tick_cnt

; =========================  WAIT_TICKS (ESPERAR TICKS)
   ==============================
;  Espera por um numero de "ticks" do FED especificado em r0 (0...65535)
   .
;  Utiliza e modifica os registadores r1 e r2.
; ---------------------------------------------------------------------
wait_ticks:
    ldr     r1, _tick_ptr2                  ; r1 = endereco da variavel
   tick_cnt
    ldr     r2, [r1]                        ; r2 = valor atual de
   tick_cnt (tempo inicial t0)
    add     r0, r0, r2                      ; r0 = tempo alvo (t0 +
   atraso desejado em r0)
WT_loop:                                    ; Loop de espera ativa
    ldr     r2, [r1]                        ; r2 = valor atual de
   tick_cnt
    cmp     r2, r0                          ; Compara o tick_cnt atual
   com o tempo alvo
    blo     WT_loop                         ; Se atual < alvo (unsigned)
   , continua no loop
    mov     pc,  lr                         ; Retorna da sub-rotina
_tick_ptr2:
    .word   tick_cnt                        ; Literal que armazena o
   endereco da variavel tick_cnt

; ==========================  MAIN (ROTINA PRINCIPAL)
   ==================================
main:
    push    lr                              ; Salva o registo de link (
```

```
    endereco de retorno)
     push    r4                              ; Salva r4 (usado para
    indice do dado)
     push    r5                              ; Salva r5 (usado para
    estado anterior do botao ROLL)

     mrs     r0, cpsr                        ; Le o registo de estado do
    programa (CPSR) para r0
     mov     r1, #ENABLE_EXTINT              ; Carrega o bit de
    habilitacao de interrupcao externa em r1
     orr     r0, r0, r1                      ; Define o bit de
    habilitacao de interrupcao no valor lido do CPSR
     msr     cpsr, r0                        ; Escreve o novo valor no
    CPSR para habilitar interrupcoes externas
     b       main_init                       ; Salta para a rotina de
    inicializacao

main_init:                                   ; Rotina de inicializacao do
     programa
     bl      inport_read                     ; Le o estado dos DIP-
    switches (r0 = valor lido)
     bl      srand                           ; Inicializa o gerador de
    numeros aleatorios (semente em r0 e r1, aqui r1 e implicitamente 0)
     bl      roll                            ; Faz um lancamento inicial
    (r0 = indice do dado, resultado em r0)

     ldr     r1, ptr_hex27seg                ; r1 <- endereco da tabela
    de conversao para display de 7 segmentos
     ldrb    r0, [r1, r0]                    ; r0 <- codigo do display
    para o digito (resultado do roll)
     bl      outport_write                   ; Escreve o digito no
    display de 7 segmentos

     mov     r4, #0                          ; r4 = indice do dado atual
    (inicializado como 0)
     mov     r5, #0                          ; r5 = estado anterior do
    botao ROLL (0 = nao premido/estado inicial)
     mov     r0, #0                          ; r0 = estado atual do botao
     ROLL (para o primeiro loop)

main_start_loop:                             ; Loop inicial, aguarda o
    primeiro pressionar de ROLL
     mov     r5, r0                          ; Salva o estado anterior do
     botao ROLL (r5 = estado_anterior_roll)
     mov     r0, r4                          ; r0 = indice do dado atual
```

```
   (para a funcao action)
    bl      action                          ; Verifica se o tipo de dado
    mudou nos DIPs e atualiza o display
    mov     r4, r0                          ; r4 = novo indice do dado (
   retornado por action)
    ; Espera que o botao ROLL seja premido (transicao de nao premido
   para premido)
    bl      inport_read                     ; r0     estado dos DIP-
   switches
    mov     r1, #ROLL_MSK                   ; r1 = mascara para o botao
   ROLL
    and     r0, r0, r1                      ; Isola o bit do botao ROLL
   em r0 (0 se premido, 1 se solto, assumindo logica do switch)

    cmp     r0, r5                          ; Compara estado atual de
   ROLL (r0) com o anterior (r5)
    bhs     main_start_loop                 ; Se r0 >= r5 (nao houve
   transicao 1->0, ou seja, nao foi premido ou continua premido/solto),
   continua
    b       Main_loop                       ; Se r0 < r5 (transicao
   1->0, botao foi premido), inicia o loop principal

; -------- Loop principal do jogo
   ---------------------------------------------
Main_pre_loop:                              ; Ponto de espera para o
   proximo pressionar de ROLL
    mov     r5, r0                          ; Salva o estado atual do
   botao ROLL (que e "premido" ou o estado que saiu do loop anterior)

Main_loop:
    ; Espera que o botao ROLL seja premido novamente
    bl      inport_read                     ; r0     estado dos DIP-
   switches
    mov     r1, #ROLL_MSK                   ; r1 = mascara para o botao
   ROLL
    and     r0, r0, r1                      ; Isola o bit do botao ROLL
   em r0

    cmp     r0, r5                          ; Compara estado atual de
   ROLL (r0) com o anterior (r5)
    bhs     Main_pre_loop                   ; Se ROLL nao mudou para "
   premido" (transicao 1->0), volta a Main_pre_loop

    mov     r5, r0                          ; Atualiza o estado anterior
    de ROLL para o estado atual ("premido")
```

```
    bl      inport_read                     ; Le novamente os DIP-
   switches para obter a selecao do dado
    mov     r6, #DICE_MSK                   ; r6 = mascara para os bits
   de selecao do dado
    and     r0, r0, r6                      ; Isola os bits de selecao
   do dado em r0
    lsr     r0, r0, #2                      ; Desloca para a direita 2
   bits para obter o indice do dado (0-3)
    mov     r6, r0                          ; r6     indice do dado
   (0-3)

    ; Executa a animacao
    bl      anim                            ; Chama a rotina de animacao

    ; Gera uma face aleatoria para o dado selecionado e mostra o
   resultado
    mov     r0, r6                          ; r0     indice do dado
   (0-3) para a funcao roll
    bl      roll                            ; Chama a rotina de
   lancamento do dado (resultado em r0)

    ldr     r1, ptr_hex27seg                ; r1     endereco da tabela
   de conversao para 7 segmentos
    ldrb    r0, [r1, r0]                    ; r0     codigo do display
   para o digito resultante
    bl      outport_write                   ; Escreve o digito no
   display

    mov     r0, #END_ROLL & 0xFF            ; Carrega a parte baixa da
   constante END_ROLL em r0
    movt    r0, #END_ROLL >> 8              ; Carrega a parte alta da
   constante END_ROLL em r0
    bl      wait_ticks                      ; Espera por END_ROLL "ticks
   " para mostrar o resultado

    b       Main_loop                       ; Volta ao inicio do loop
   principal para aguardar novo lancamento

; =====================  ACTION (AcaO DE ATUALIZAcaO DO DISPLAY)
   ====================================
; Descricao: Verifica se o tipo de dado selecionado no DIP-switch mudou.
;            Se mudou, faz um novo "lancamento" (mostra '1' por defeito
   ou face aleatoria, dependendo de 'roll')
;            e atualiza o display.
```

```
; Entrada:   r0 = indice do tipo de dado anterior (0-3)
; Saida:     r0 = indice do tipo de dado atual (0-3), lido dos DIPs.
; Modifica:  r1, r5 (r4 e usado internamente mas restaurado ao valor de
   entrada se nao houver mudanca)
; ---------------------------------------------------------------------
action:
    push    lr                              ; Salva o registo de link
    push    r4                              ; Salva r4 (indice anterior)
    push    r5                              ; Salva r5 (usado para
   mascara)

    mov     r4, r0                          ; r4 = indice do dado
   anterior (passado como argumento em r0)
    bl      inport_read                     ; Le o estado dos DIP-
   switches para r0
    mov     r5, #DICE_MSK                   ; r5 = mascara para os bits
   de selecao do dado
    and     r0, r0, r5                      ; Isola os bits de selecao
   do dado em r0
    lsr     r0, r0, #2                      ; Desloca para a direita
   para obter o novo indice do dado (0-3)
    cmp     r0, r4                          ; Compara o novo indice (r0)
    com o anterior (r4)
    beq     Action_ret                      ; Se forem iguais, nao faz
   nada e retorna o indice atual (que e o mesmo)

    ; Se o indice do dado mudou:
    mov     r4, r0                          ; r4 = novo indice do dado (
   para ser usado no roll e retornado)
    bl      roll                            ; Faz um lancamento para o
   novo tipo de dado (r0 = novo indice, saida r0 = resultado do dado)

    ldr     r1, ptr_hex27seg                ; r1     endereco da tabela
   de conversao para 7 segmentos
    ldrb    r0, [r1, r0]                    ; r0     codigo do display
   para o digito resultante do 'roll'
    bl      outport_write                   ; Escreve o novo digito no
   display

    mov     r0, r4                          ; Prepara o valor de retorno
   : r0 = novo indice do dado
Action_ret:
    pop     r5                              ; Restaura r5
    pop     r4                              ; Restaura r4
    pop     pc                              ; Retorna da sub-rotina (pc
```

```
   = lr)

ptr_hex27seg:
    .word   hex27seg                        ; Literal que armazena o
   endereco da tabela hex27seg

; =====================  ROLL (LANcAR DADO)
   ========================================
; Descricao: Gera um numero aleatorio para o tipo de dado especificado.
; Entrada:   r0 = indice do tipo de dado (0-3, correspondendo a d4, d6,
   d8, etc.)
; Saida:     r0 = numero resultante do lancamento (1 a N, onde N e o
   numero de faces do dado)
; Modifica:  r1, r2, r3 (usados por rand), r4 (usado localmente para N)
; ---------------------------------------------------------------------
roll:
    push    lr                              ; Salva o registo de link
    push    r4                              ; Salva r4 (sera usado para
   N, o numero de faces)
    bl      getDiceMax                      ; Chama getDiceMax: r0 (
   indice) -> r0 (N = numero de faces)
    mov     r4, r0                          ; r4 = N (numero de faces do
    dado, o divisor para o modulo)

    bl      rand                            ; Chama o gerador de numeros
    aleatorios (resultado de 16 bits em r0, 0-65535)
    lsr     r0, r0, #8                      ; Reduz o intervalo do
   numero aleatorio para 0-255 (r0 = r0 / 256)
                                            ; Isto e feito para
   simplificar a operacao de modulo com N (tipicamente pequeno, <=12).

; ---------- Calculo: r0 = (r0 mod N) + 1  (onde N esta em r4)
   ------------------
; Implementacao do modulo por subtracoes sucessivas para obter um
   resultado em [0, N-1].
mod_loop:
    cmp     r0, r4                          ; Compara r0 com N (em r4)
    blo     mod_done                        ; Se r0 < N, entao r0 ja e o
    resto (0 a N-1), termina o loop
    sub     r0, r0, r4                      ; r0 = r0 - N
    ; blo mod_done                          ; Este 'blo' adicional e
   geralmente redundante se N > 0 e r0 inicial >= 0.
                                            ; Se r0 se tornou < N (mas
   nao negativo), o primeiro 'blo' ja teria ramificado.
                                            ; Se N e muito grande e r0
```

```
   pequeno, sub pode dar underflow se nao for unsigned.
                                          ; Assumindo r0 e r4 sao
   positivos, o primeiro blo e suficiente.
    b       mod_loop                      ; Continua no loop
mod_done:
    add     r0, r0, #1                    ; Ajusta o resultado para
   ser 1-based (1 a N) em vez de 0-based (0 a N-1)

    pop     r4                            ; Restaura r4
    pop     pc                            ; Retorna da sub-rotina

; ------------ getDiceMax (OBTER MaXIMO DE FACES DO DADO)
   ---------------------------------------------
; ------------------------------------------------------------
; Descricao: Obtem o numero maximo de faces para um dado, com base no
   seu indice.
; Entrada:
;   r0 - indice do dado (0 para d4, 1 para d6, 2 para d8, 3 para d12 -
   conforme `max_sides`)
; Retorna:
;   r0 - Numero maximo de faces para o dado selecionado.
;        Se o indice (r0) for >= TOTAL_SIDES, r0 e retornado inalterado.
; Modifica: r4 (usado localmente)
getDiceMax:
    push    r4                            ; Salva r4 na pilha
    mov     r4, #TOTAL_SIDES              ; r4 = Numero total de tipos
    de dados diferentes (ex: 4)
    cmp     r0, r4                        ; Compara o indice do dado (
   r0) com TOTAL_SIDES
    bge     getDiceMax_end                ; Se indice (r0) >=
   TOTAL_SIDES, salta para o fim (indice invalido, retorna r0 como esta)
    lsl     r0, r0, #1                    ; Multiplica o indice por 2
   (r0 = r0 * 2), pois cada entrada em `max_sides` e uma .word (2 bytes)
    ldr     r4, ptr_max_sides             ; r4 = endereco base da
   tabela `max_sides`
    ldr     r0, [r4, r0]                  ; Carrega o valor da tabela:
    r0 = max_sides[indice_original*2] (le uma palavra de 16 bits)
getDiceMax_end:
    pop     r4                            ; Restaura r4
    mov     pc, lr                        ; Retorna da sub-rotina

ptr_max_sides:
    .word   max_sides                     ; Literal que armazena o
   endereco da tabela max_sides
```

```
; ======================  ANIMAcaO ====================================
; Descricao: Executa uma animacao no display de 7 segmentos.
;            Mostra FRAMES_ROLL frames, cada um com duracao de
   FRAME_TICKS "ticks".
; Modifica:  r0, r1 (para wait_ticks e outport_write)
; Preserva:  lr, r4, r5, r6 (via pilha)
; ---------------------------------------------------------------------
anim:
    push    lr                              ; Salva o registo de link
    push    r4                              ; Salva r4 (indice do padrao
    de animacao)
    push    r5                              ; Salva r5 (contador de
   frames da animacao)
    push    r6                              ; Salva r6 (ponteiro para a
   sequencia de animacao)

    ldr     r6, ptr_anim_seq                ; r6 = endereco base da
   sequencia de animacao (`anim_sequence`)
    mov     r4, #0                          ; r4 = indice (offset em
   bytes) do padrao atual na sequencia de animacao
    mov     r5, #FRAMES_ROLL & 0xFF         ; Carrega a parte baixa de
   FRAMES_ROLL em r5 (contador de frames totais a exibir)
    movt    r5, #FRAMES_ROLL >> 8           ; Carrega a parte alta de
   FRAMES_ROLL em r5

Anim_loop:                                  ; Loop principal da animacao
    mov     r0, #FRAME_TICKS                ; r0 = duracao de um frame
   em "ticks"
    bl      wait_ticks                      ; Espera pela duracao de um
   frame

    ldrb    r0, [r6, r4]                    ; Carrega o byte do padrao
   de animacao atual em r0 (de anim_sequence[r4])
                                            ; (Assume que anim_sequence
   armazena padroes de byte, e r4 e o offset)
    bl      outport_write                   ; Mostra o padrao no display
    de 7 segmentos

    add     r4, r4, #2                      ; Avanca para o proximo
   padrao na sequencia (incrementa o offset por 2 bytes,
                                            ; pois `anim_sequence` e
   definida com `.word`, e estamos lendo o LSB de cada word)
    mov     r1, #(MAX_ANIM * 2)             ; r1 = tamanho total em
   bytes da tabela de animacao (MAX_ANIM entradas de 2 bytes cada)
    cmp     r4, r1                          ; Compara o indice atual com
```

```
    o tamanho total
    blt     Anim_idx_ok                     ; Se indice < tamanho, esta
   dentro dos limites da tabela
    mov     r4, #0                          ; Se atingiu o fim da
   sequencia, volta ao inicio (indice = 0)
Anim_idx_ok:

    sub     r5, r5, #1                      ; Decrementa o contador de
   frames restantes
    bne     Anim_loop                       ; Se ainda ha frames para
   mostrar (r5 != 0), continua o loop

    pop     r6                              ; Restaura r6
    pop     r5                              ; Restaura r5
    pop     r4                              ; Restaura r4
    pop     pc                              ; Retorna da sub-rotina

ptr_anim_seq:
    .word   anim_sequence                   ; Literal que armazena o
   endereco da sequencia de animacao

; =====================  ROTINAS DE E/S (ENTRADA/SAiDA)
   ================================
;-------------------------------------------------------
; Rotinas da Porta de Entrada
;-------------------------------------------------------
inport_read:                                ; Le um byte da porta de
   entrada
                                            ; Saida: r0 = valor lido da
   porta de entrada
                                            ; Modifica: r1
    mov     r1, #INPORT_ADDRESS & 0xFF      ; Carrega a parte baixa do
   endereco da porta de entrada em r1
    movt    r1, #(INPORT_ADDRESS >> 8) & 0xFF ; Carrega a parte alta do
   endereco da porta de entrada em r1
    ldrb    r0, [r1, #0]                    ; Le um byte do endereco em
   r1 e armazena em r0
    mov     pc, lr                          ; Retorna da sub-rotina

;-------------------------------------------------------
; Rotinas da Porta de Saida
;-------------------------------------------------------
outport_write:                              ; Escreve um byte na porta
   de saida
                                            ; Entrada: r0 = valor a ser
```

```
   escrito na porta de saida
                                                ; Modifica: r1
    mov     r1, #OUTPORT_ADDRESS & 0xFF         ; Carrega a parte baixa do
   endereco da porta de saida em r1
    movt    r1, #(OUTPORT_ADDRESS >> 8) & 0xFF; Carrega a parte alta do
   endereco da porta de saida em r1
    strb    r0, [r1, #0]                        ; Escreve o byte de r0 no
   endereco em r1
    mov     pc, lr                              ; Retorna da sub-rotina

; Rotina:    umull32
; Descricao: Realiza a multiplicacao de dois numeros naturais de 32 bits
    (M * m).
;            Retorna os 32 bits menos significativos do produto de 64
   bits.
;            A implementacao parece ser baseada numa variacao do
   algoritmo de Booth ou similar.
; Interface (conceptual): uint32_t umull32( uint32_t M_high:M_low,
   uint32_t m_high:m_low );
; Entradas:  R1:R0 - Multiplicando (M) de 32 bits (R1=MSW, R0=LSW)
;            R3:R2 - Multiplicador (m) de 32 bits (R3=MSW, R2=LSW)
; Saidas:    R1:R0 - 32 bits menos significativos do produto (P = M * m)
    (R1=MSW, R0=LSW)
; Modifica:  R4, R5, R6, R7, R8 (alem de R0-R3 que formam o resultado)
;
umull32:
    ; Prologo
    push    r8                                  ; Salva os registadores
   utilizados na rotina
    push    r7
    push    r6
    push    r5
    push    r4

    ; Inicializacao de 'p' (acumulador do produto, R5:R4 para parte alta
   ) e 'p_1' (R6, bit auxiliar)
    ; A inicializacao de R5:R4 com `asr r4, r3, #15` e atipica para
   multiplicacao de "numeros naturais" (sem sinal).
    ; Para multiplicacao sem sinal, R5:R4 seriam usualmente
   inicializados a 0.
    ; Esta inicializacao sugere um algoritmo para numeros com sinal ou
   uma variante especifica.
    asr     r4, r3, #15                         ; R4 = Extensao de sinal do
   MSB de R3 (parte alta do multiplicador)
    mov     r5, r4                              ; R5 = R4 (R5:R4
```

```
   inicializado com a extensao de sinal)
    mov     r6, #0                          ; R6 (p_1, bit auxiliar) = 0
    mov     r7, #0                          ; R7 (contador de iteracao i
   ) = 0
umull32_loop:
    mov     r8, #32                         ; Limite de 32 iteracoes
   para multiplicacao de 32 bits
    cmp     r7, r8                          ; Compara i com 32
    bhs     umull32_ret                     ; Se i >= 32, termina o loop
    ; Logica condicional baseada no LSB de R2 (parte baixa do
   multiplicador) e R6 (p_1)
    mov     r8, #1
    and     r8, r2, r8                      ; Verifica o LSB de R2 (m_i)
    bzc     umull32_else                    ; Se LSB de R2 (m_i) e 0,
   salta para 'else'
    ; Caso LSB de R2 (m_i) seja 1:
    mov     r8, #1
    cmp     r6, r8                          ; Compara p_1 (R6) com 1 (m_
   {i-1})
    bne     umull32_loop_end                ; Se p_1 != 1 (transicao 01
   ou 11, onde p_1 e m_{i-1}), nao faz adicao/subtracao aqui
                                            ; (Parece ser: se m_i=1 e m_
   {i-1}=1, P = P + M)
    add     r4, r4, r0                      ; P_low = P_low + M_low
    adc     r5, r5, r1                      ; P_high = P_high + M_high +
    carry
    b       umull32_loop_end
umull32_else:
    ; Caso LSB de R2 (m_i) seja 0:
    mov     r8, #0
    cmp     r6, r8                          ; Compara p_1 (R6) com 0 (m_
   {i-1})
    bne     umull32_loop_end                ; Se p_1 != 0 (transicao 10
   ou 00, onde p_1 e m_{i-1}), nao faz adicao/subtracao aqui
                                            ; (Parece ser: se m_i=0 e m_
   {i-1}=0, P = P - M)
    sub     r4, r4, r0                      ; P_low = P_low - M_low
    sbc     r5, r5, r1                      ; P_high = P_high - M_high -
    borrow
umull32_loop_end:
    ; Atualiza p_1 (R6) e faz o shift aritmetico a direita do acumulador
    (R5:R4) e do multiplicador (R3:R2)
    mov     r8, #1
    and     r6, r2, r8                      ; Novo p_1 (R6) = LSB de R2
   (m_i, antes do shift)
```

```
    asr     r5, r5, #1                      ; Shift aritmetico a direita
    de P_high (R5)
    rrx     r4, r4                          ; Rotaciona P_low (R4) a
   direita com carry do ASR de R5
    rrx     r3, r3                          ; Rotaciona m_high (R3) a
   direita com carry do RRX de R4
    rrx     r2, r2                          ; Rotaciona m_low (R2) a
   direita com carry do RRX de R3
                                            ; O conjunto R5:R4:R3:R2 e
   efetivamente shiftado a direita como uma unidade de 64 bits.
    add     r7, r7, #1                      ; Incrementa o contador de
   iteracao i
    b       umull32_loop

umull32_ret:
    ; Epilogo
    mov     r0, r2                          ; Prepara o valor de retorno
   : R0 = R2 (LSW do produto de 32 bits)
    mov     r1, r3                          ; R1 = R3 (MSW do produto de
    32 bits)

    pop     r4                              ; Restaura os registadores
   salvos
    pop     r5
    pop     r6
    pop     r7
    pop     r8
    mov     pc, lr                          ; Retorna da sub-rotina

; Rotina:    srand
; Descricao: Define o valor inicial (semente) para o gerador de numeros
   pseudo-aleatorios (LCG).
; Interface (conceptual): void srand( uint32_t nova_semente_high:
   nova_semente_low );
; Entradas:  R0 - Parte baixa (bits 0-15) da nova semente.
;            R1 - Parte alta (bits 16-31) da nova semente.
; Saidas:    Nenhuma.
; Efeitos:   Altera o valor da variavel global `seed`.
; Modifica:  R2 (usado para endereco).
;
srand:
    ldr     r2, seed_addr_srand             ; R2 = endereco da variavel
   global `seed`
    str     r0, [r2, #0]                    ; Armazena R0 (nova
   semente_low) em seed[0..15]
```

```
    str     r1, [r2, #2]                    ; Armazena R1 (nova
   semente_high) em seed[16..31]
    mov     pc, lr                          ; Retorna da sub-rotina

seed_addr_srand:
    .word   seed                            ; Literal que armazena o
   endereco da variavel `seed`

; Rotina:    rand
; Descricao: Implementa um gerador congruencial linear (LCG).
;            Formula: seed = (seed * 0x000343FD + 0x00269EC3) mod 2^32.
;            Retorna os 16 bits mais significativos da nova semente.
; Interface (conceptual): uint16_t rand( void );
; Entradas:  Nenhuma (utiliza a semente global `seed`).
; Saidas:    R0 - Valor pseudo-aleatorio de 16 bits gerado (MSBs da nova
    semente).
; Efeitos:   Altera o valor da variavel global `seed`.
; Modifica:  R0, R1, R2, R3 para calculos internos.
;
rand:
    ; Prologo
    push    lr                              ; Salva o registo de link
    ; Obter o valor atual da semente (seed) de 32 bits (R1:R0)
    ldr     r2, seed_addr_rand              ; R2 = endereco de `seed`
    ldr     r0, [r2, #0]                    ; R0 = seed_low (parte baixa
    da semente atual)
    ldr     r1, [r2, #2]                    ; R1 = seed_high (parte alta
    da semente atual)

    ; Calcular a multiplicacao: (seed * multiplicador), onde
   multiplicador = 0x000343FD (214013)
    mov     r2, #( 0x43FD >> 0 ) & 0xFF     ; R2 = parte baixa do
   multiplicador (0x43FD)
    movt    r2, #( 0x43FD >> 8 ) & 0xFF
    mov     r3, #( 0x0003 >> 0 ) & 0xFF     ; R3 = parte alta do
   multiplicador (0x0003)
    movt    r3, #( 0x0003 >> 8 ) & 0xFF
    bl      umull32                         ; (R1:R0) = (R1:R0) * (R3:R2
   ). Resultado da multiplicacao em R1:R0.

    ; Calcular a adicao: (resultado_multiplicacao + incremento), onde
   incremento = 0x00269EC3 (2531011)
    mov     r2, #( 0x9EC3 >> 0 ) & 0xFF     ; R2 = parte baixa do
   incremento (0x9EC3)
    movt    r2, #( 0x9EC3 >> 8 ) & 0xFF
```

```
    mov     r3, #( 0x0026 >> 0 ) & 0xFF      ; R3 = parte alta do
   incremento (0x0026)
    movt    r3, #( 0x0026 >> 8 ) & 0xFF
    add     r0, r0, r2                       ; Adiciona a parte baixa do
   incremento a parte baixa do resultado (R0)
    adc     r1, r1, r3                       ; Adiciona a parte alta do
   incremento a parte alta (R1), com carry

    ; Tratamento especial: se a nova semente (R1:R0) for 0xFFFFFFFF (
   RAND_MAX), forca-a para 0.
    ; Esta e uma modificacao ao comportamento padrao de um LCG.
    mov     r2, #( RAND_MAX_L >> 0 ) & 0xFF ; R2 = 0xFFFF (RAND_MAX_L)
    movt    r2, #( RAND_MAX_L >> 8 ) & 0xFF
    cmp     r0, r2                           ; Compara R0 (nova seed_low)
    com 0xFFFF
    bne     rand_save_seed                   ; Se R0 != 0xFFFF, nao e
   RAND_MAX, entao salva a semente calculada
    mov     r3, #( RAND_MAX_H >> 0 ) & 0xFF ; R3 = 0xFFFF (RAND_MAX_H)
    movt    r3, #( RAND_MAX_H >> 8 ) & 0xFF
    cmp     r1, r3                           ; Compara R1 (nova seed_high
   ) com 0xFFFF
    bne     rand_save_seed                   ; Se R1 != 0xFFFF, nao e
   RAND_MAX, entao salva a semente calculada
    ; Se R1:R0 == 0xFFFFFFFF:
    mov     r0, #0                           ; Forca a nova semente para
   0 (seed_low = 0)
    mov     r1, #0                           ; seed_high = 0

rand_save_seed:
    ; Atualizar o valor da semente global ‘seed‘ com o novo valor em R1:
   R0
    ldr     r2, seed_addr_rand               ; R2 = endereco de ‘seed‘
    str     r0, [r2, #0]                     ; Salva a nova seed_low
    str     r1, [r2, #2]                     ; Salva a nova seed_high

    ; Epilogo
    mov     r0, r1                           ; Prepara o valor de retorno
   : R0 = R1 (os 16 MSBs da nova semente de 32 bits)
    pop     pc                               ; Retorna da sub-rotina

seed_addr_rand:
    .word   seed                             ; Literal que armazena o
   endereco da variavel ‘seed‘
    ; Definicao da variavel global ‘seed‘ (tipo uint32_t) com valor
   inicial 1.
```

```
    ; O processador P16 utiliza ordenacao little-endian: o byte menos
   significativo (LSB)
    ; e armazenado no endereco de memoria mais baixo.
    ; O valor 1 (0x00000001) e armazenado como:
    ;   seed+0 (LSW byte 0): 0x01
    ;   seed+1 (LSW byte 1): 0x00
    ;   seed+2 (MSW byte 0): 0x00
    ;   seed+3 (MSW byte 1): 0x00
seed:
    .word   1, 0                            ; seed_low = 1 (0x0001),
   seed_high = 0 (0x0000). Valor inicial da semente = 0x00000001.

; ============================  DADOS ==================================
    .data
    .align  2                               ; Alinha os dados seguintes
   em enderecos multiplos de 2 (para .word)

; Ponteiros constantes (literais) para enderecos. uteis se o
   enderecamento PC-relativo for limitado.
inport_addr:    .word   INPORT_ADDRESS      ; Endereco da porta de
   entrada (nao usado diretamente no codigo, INPORT_ADDRESS e usado)
outport_addr:   .word   OUTPORT_ADDRESS     ; Endereco da porta de saida
    (nao usado diretamente no codigo, OUTPORT_ADDRESS e usado)

tick_cnt_ptr:   .word   tick_cnt            ; Ponteiro para a variavel
   tick_cnt (usado por _tick_ptr1 e _tick_ptr2)
rng_seed:       .word   12345               ; Semente de RNG (nao usada
   pela logica atual de `srand` que le dos DIPs). Pode ser para teste/
   depuracao.

tick_cnt:       .word   0                   ; Contador de "ticks" do
   sistema, incrementado pela ISR da interrupcao externa.
max_sides:      .word   4, 6, 8, 6          ; Tabela com o numero de
   faces para cada tipo de dado.
                                            ; indice 0: 4 faces (d4)
                                            ; indice 1: 6 faces (d6)
                                            ; indice 2: 8 faces (d8)
                                            ; indice 3: 6 faces (d6
   novamente. Para um d12, seria 12 aqui.)
    .align  1                               ; Alinha os dados seguintes
   em enderecos multiplos de 1 (byte)

anim_sequence:                              ; Sequencia de padroes para
   animacao no display de 7 segmentos.
                                            ; Cada `.word` armazena um
```

```
   padrao de byte no seu LSB, acedido por ‘ldrb‘.
    .word   0x21                            ; Padrao de animacao 0 (ex:
   segmento ’a’ e ’f’)
    .word   0x03                            ; Padrao de animacao 1 (ex:
   segmento ’a’ e ’b’)
    .word   0x42                            ; ... e assim por diante
   para os 8 padroes (MAX_ANIM)
    .word   0x50
    .word   0x18
    .word   0x0C
    .word   0x44
    .word   0x60

hex27seg:                                   ; Tabela de mapeamento de
   valores numericos (0-15) para codigos de display de 7 segmentos.
                                            ; Formato do byte: (dp)
   gfedcba, onde dp e o ponto decimal (nao usado aqui), g e MSB.
    .byte   0x3F, 0x06, 0x5B, 0x4F, 0x66, 0x6D, 0x7D, 0x07  ; Digitos 0,
    1, 2, 3, 4, 5, 6, 7
    .byte   0x7F, 0x6F, 0x77, 0x7C, 0x39, 0x5E, 0x79, 0x71  ; Digitos 8,
    9, A, b, C, d, E, F (representacoes hexadecimais)

; ==========================  PILHA (STACK)
   ==================================
    .stack                                  ; Define a seccao da pilha
   na memoria
    .space  STACK_SIZE                      ; Reserva STACK_SIZE bytes
   para a pilha
stack_top:                                  ; Rotulo que marca o topo
   inicial da pilha (endereco mais alto da area da pilha)
                                            ; A pilha cresce para
   enderecos mais baixos.
```

Código 1: Implementacao do programa, em *assembly* do P16, proposta por nos